TU Clausthal

Master's Thesis

by

Yu Zhang

SimTutor: A Telemetry-Grounded Tutoring Runtime with Structured Visual Fact Extraction for Procedural Learning

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Prof. Dr. Rüdiger Ehlers**

9th May 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 9th May 2026	
Location, Date	Yu Zhang

Abstract

Zusammenfassung

Acknowledgements

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

1.2 Thesis Vision and Scope

1.3 Current Implementation Scope

1.4 Contributions

1.5 Thesis Structure

2 Background and Related Work

This chapter provides the domain and technical background necessary to understand the design of the SimTutor system, followed by a survey of related work across several intersecting research areas. Section 2.1 introduces the F/A-18C cold-start task that serves as the training scenario for the system. Section 2.2 describes the visual fact ontology, a structured intermediate representation that bridges raw cockpit screenshots and downstream procedural reasoning. The remainder of the chapter positions the present work against existing research in intelligent tutoring systems, simulator-based training, state-aware tutoring, vision-language model adaptation, and grounded AI assistance.

2.1 The F/A-18C Cold-Start Task

The procedural training scenario at the centre of this thesis is the F/A-18C cold-start, a standard pre-flight cockpit procedure that brings the aircraft from a powered-down state to a taxi-ready configuration. This task is canonical in DCS World training curricula and is representative of a broader class of fixed-sequence, state-sensitive procedural tasks in which the order of actions, the verification of intermediate system states, and the correct interpretation of cockpit instrumentation all contribute to successful completion.

2.1.1 Procedure Structure: S01–S25 and Phases P1–P6

The cold-start procedure is organised into 25 numbered steps (S01–S25), grouped into six coarse-grained phases (P1–P6) that reflect distinct operational stages:

P1 — Power-up and safety (S01–S03): Battery and generator activation, fire detection system tests, and auxiliary power unit (APU) start. These steps establish electrical power and confirm critical safety systems are functional before engine ignition.

P2 — Right engine start (S04–S07): Right engine crank, throttle advance at 25% RPM, bleed air system cycling, and caution/warning/advisory lights testing. The

right engine is started first to provide hydraulic and electrical power for subsequent procedures.

P3 — Displays and communications (S08–S09): Power-up of both Digital Display Indicators (DDIs), the Advanced Multipurpose Color Display (AMPCD), and the Head-Up Display (HUD); configuration of specific display pages (FCS page on left DDI, BIT root page on right DDI); and programming of COMM1 preset frequencies via the Up-Front Controller (UFC).

P4 — Left engine and core systems (S10–S14): Left engine start following verification of right-engine parameters in nominal ranges, INS alignment mode selection (ground or carrier), radar power-up, and OBOGS (On-Board Oxygen Generating System) activation.

P5 — FCS and flight controls (S15–S19): Flight Control System (FCS) reset and BIT (Built-In Test) execution, flap and trim configuration, and the "four-down" pre-flight checks covering the refuelling probe, speed brake, launch bar, and arrestor hook.

P6 — Pre-taxi instruments and final checks (S20–S25): Parking brake release, BINGO fuel setting, barometric and radar altimeter configuration, standby attitude indicator uncaging, and attitude source selection.

The procedure is strictly ordered: each step carries explicit precondition gates that depend on the completion of earlier steps and on the satisfaction of specific telemetry conditions. For example, S04 (engine crank right) requires that the APU start support has been established (S03 completion), while S10 (engine crank left) requires that right-engine parameters — RPM, temperature, fuel flow, nozzle position, and oil pressure — all fall within their nominal green ranges before the left engine is engaged. These inter-step dependencies make the cold-start task a compelling test case for context-aware tutoring: a learner who skips a verification or misjudges an instrument reading can propagate errors through the remainder of the sequence.

2.1.2 Cockpit Displays and the Composite Panel

The visual fact extraction subsystem (ğ2.2) operates on a fixed composite panel image that captures three cockpit display regions, arranged from top to bottom:

- **Left DDI (Digital Display Indicator):** A multi-function display used for the FCS page, SUPT (Setup) page, TAC (Tactical) page, and other system pages. During the cold-start, the left DDI is primarily used for FCS monitoring (S08, S15).
- **AMPCD (Advanced Multipurpose Color Display):** The central colour display, used for the HSI (Horizontal Situation Indicator) page, INS alignment status, map layers, and navigation data. In the cold-start, the AMPCD becomes critical during the INS alignment phase (S12).
- **Right DDI:** A second multi-function display, used during the cold-start for the BIT root page (S08) and later for the FCS-MC BIT sequence (S18). The right DDI is also where BIT failure indications and FCS BIT intermediate and final results appear.

The composite panel image consolidates these three regions into a single input for the vision-language model (VLM). This design choice avoids mismatches between training and inference (where the model would be trained on per-region crops but evaluated on a full composite, or vice versa) and simplifies the multi-turn visual pipeline to a single-image extraction step per observation window [1].

2.1.3 DCS-BIOS Telemetry

DCS-BIOS provides a real-time UDP-based telemetry stream from DCS World, exporting the state of cockpit switches, indicators, warning lights, gauges, and other instruments as structured data. The SimTutor runtime receives these raw BIOS frames and passes them through a telemetry enrichment pipeline that normalises the raw values into stable runtime variables (e.g., `rpm_r`, `apu_ready`, `ins_mode`), applies delta sanitisation to suppress noise and jitter, and latches discrete state-completion flags for gate evaluation.

This telemetry feed is the primary grounding signal for the tutoring system. Unlike vision-only approaches, which must infer procedure state from pixel-level evidence alone, the SimTutor runtime can cross-reference visual observations against deterministic telemetry state. Steps such as S01 (battery and generators), S05 (right throttle idle), and S13 (radar mode OPR) are directly observable via DCS-BIOS and require no visual inference. Other steps, such as S08 and S18, involve display-page state that is not directly exported via DCS-BIOS and therefore rely on the visual fact extraction pipeline

described below.

2.2 Visual Fact Ontology

A central design decision in the SimTutor system is the separation between visual perception and procedural reasoning. Rather than training a model to directly produce a tutor decision ("the learner should now press PB5"), the system uses an intermediate structured representation called *visual facts*. A visual fact is a discrete, auditable proposition about the visible state of one cockpit display region at a single point in time. Each fact is labelled with one of three states:

`seen`: The visual evidence described by the fact is clearly visible in the current cockpit image.

`not_seen`: The visual evidence is absent or not discernible.

`uncertain`: The image is blurry, occluded, incomplete, or otherwise insufficient for a confident determination.

These facts are consumed downstream by the procedure engine (ğ**??**), which combines them with telemetry state, gating rules, and knowledge retrieval to decide whether a procedure step is visually confirmed, still pending, or blocked. This layered architecture isolates the VLM's responsibility to structured visual evidence extraction, reserving procedure-level decisions for deterministic rule-based components.

2.2.1 From 8-Fact v1 to 13-Fact v2

The visual fact ontology evolved across two generations, reflecting lessons from early fine-tuning experiments and the needs of the runtime procedure pack.

8-fact v1 (early baseline). The first-generation ontology contained eight facts targeting the three display regions. These facts covered FCS page visibility (left DDI), BIT page and BIT failure visibility (right DDI), FCS-MC page and in-test status (right DDI), FCS BIT result visibility (right DDI), INS alignment page visibility (AMPCD), and the INS GO completion signal (AMPCD). This ontology was used to train the first LoRA adapter (LoRA-v1) on 180 human-reviewed images from Run-001, yielding promising in-distribution results but exposing a critical weakness: the `ins_go` fact mixed abstraction

levels by asking the VLM to recognise a high-level procedural completion state ("INS alignment is done") from a single visual frame. The result was a high false-positive rate on the independent Run-002 holdout set (13 of 50 samples predicted `seen` when the human label was `not_seen`).

13-fact v2 (runtime-aligned). The second-generation ontology addresses this by decomposing higher-level procedure states into lower-level, visually observable evidence. The current 13-fact set is defined in `vision_facts.yaml` and includes:

- **Left DDI facts:** `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_visible`.
- **Right DDI facts:** `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_resu` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD facts:** `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_vi` `ins_ok_text_visible`.

Note that `ins_go` from v1 has been decomposed into several lower-level AMPCD facts (`ins_grnd_alignment_text_visible`, `ins_ok_text_visible`, `hsi_map_layer_visible`) that each describe a specific, locally verifiable piece of visual evidence. The downstream procedure engine, rather than the VLM, is responsible for combining these atomic observations into a procedural judgement about whether the INS alignment phase is complete.

2.2.2 Sticky Facts and Step Bindings

Two additional mechanisms in the ontology improve its robustness for runtime use.

Sticky facts. Some visual facts represent transient states that, once observed, should not be overwritten by subsequent `not_seen` frames. For example, the `fcsmc_final_go_result_visibl` fact is marked as `sticky: true` with a time-to-live of 600 seconds. Once the VLM reports this fact as `seen`, it is latched in the runtime fact state and will not be overwritten by later `not_seen` or `uncertain` predictions within the TTL window. This prevents flickering and false negatives caused by display page changes, camera occlusions, or brief rendering artifacts that occur after the critical visual event has already been captured.

Non-sticky facts (e.g., `fcs_page_visible`, `bit_root_page_visible`) carry a shorter TTL of 2 seconds and are continuously re-evaluated, reflecting the expectation that these display pages may change as the learner navigates between different cockpit pages.

Step bindings. The vision facts are linked to procedure steps through explicit step bindings defined in `vision_facts.yaml`. A step binding specifies which facts must be `seen` for the procedure engine to consider the step visually confirmed. The current ontology defines two binding types:

- `all_of`: Every fact in the binding set must be `seen`. For example, S08 requires both `fcs_page_visible` (left DDI) and `bit_root_page_visible` (right DDI) to be confirmed.
- `any_of` (one-of fact collection): At least one fact in the set must be `seen`. This accommodates cases where a procedure state can be verified through any of several alternative visual cues.

Steps that rely on visual confirmation (S08, S15, S18) are registered as `vision_priority_steps` in the procedure pack, while steps directly observable through DCS-BIOS telemetry (S01, S03–S07, S09–S13, S21) are registered as `bios_priority_steps`. Steps that require manual confirmation or involve cockpit areas outside the three-display layout (S02, S14, S16–S20, S22–S25) are marked as `manual_or_out_of_layout_steps`. This tripartite classification ensures that each step's completion evidence is sourced from the most reliable available signal.

2.3 Intelligent Tutoring Systems and Procedural Training

Intelligent Tutoring Systems (ITS) have a long history in procedural training domains, from military equipment operation to medical procedure instruction. Classic ITS architectures typically model expert knowledge, track learner state through a student model, and deliver context-sensitive feedback through a tutoring module. TODO:CITE — foundational ITS survey or architecture paper (e.g., the classic four-component ITS model or a recent comprehensive survey).

In the procedural domain, constraint-based tutors and example-tracing tutors have been applied to tasks such as aircraft maintenance, programming, and equipment troubleshooting. TODO:CITE — constraint-based modeling for procedural training; TODO:CITE — example-tracing tutors or cognitive tutors in procedural domains. These systems typically operate within closed, well-defined task environments where the space of correct and incorrect actions can be enumerated. The cold-start task studied in this thesis

shares this property of a closed procedure, but differs in two key respects. First, the system observes learner behaviour through simulator telemetry and cockpit visuals rather than through a custom tutor interface, making it applicable to unmodified simulation environments. Second, the tutoring output is generated dynamically by an LLM rather than retrieved from a pre-authored library of feedback messages, allowing the system to respond to arbitrary learner states within the procedure rather than only to anticipated error patterns.

More recent work has explored the use of large language models for tutoring and instructional dialogue. TODO:CITE — LLM-based tutoring systems or dialogue-based tutors (e.g., Khanmigo-style systems, or any LLM-as-tutor paper). These systems demonstrate that LLMs can produce fluent, contextually appropriate instructional text, but they typically operate in unstructured, conversational settings and do not enforce the strict evidence-grounding constraints that characterise the SimTutor approach. In particular, the present work constrains every overlay target and step recommendation to have a verifiable evidence source — telemetry, a gating rule, a retrieved document snippet, or a visual fact — rather than relying solely on the model's parametric knowledge.

2.4 Simulator-Based Training and Instrumentation

High-fidelity simulation environments such as DCS World provide realistic, repeatable platforms for procedural training. The use of simulators in aviation training is well established, with extensive research on transfer of training from simulated to real aircraft. TODO:CITE — simulator-based training effectiveness in aviation (e.g., transfer-of-training studies, or surveys of flight simulation for training).

A key challenge in simulator-based training research is instrumentation: the means by which a training system observes learner actions and cockpit state. Prior work has instrumented simulators through screen capture, event logging, or direct state export APIs. TODO:CITE — simulator instrumentation approaches, e.g., X-Plane data output, Microsoft Flight Simulator SimConnect, or DCS-BIOS academic usage. The present work uses DCS-BIOS, a community-developed telemetry bridge that exports DCS World cockpit state via UDP. DCS-BIOS provides a richer signal than screen capture alone, exposing the exact position of switches, the numerical values of engine parameters, and the state of warning lights without requiring computer vision to recover them. This allows the system to reserve visual perception for display-page states (which are not exported by DCS-BIOS) while relying on deterministic telemetry for switch positions

and gauge readings.

TODO:CITE — any prior academic work that uses DCS World or DCS-BIOS as a research platform for AI, HCI, or training research.

2.5 Rule-Based and State-Aware Tutoring

The SimTutor system makes extensive use of explicit gating rules, deterministic step inference, and telemetry-driven state tracking alongside its LLM-based help generation. This hybrid design places it at the intersection of rule-based and neural approaches to intelligent assistance.

Rule-based approaches to procedure tracking — using finite-state machines, petri nets, or production rules to model task progress — offer strong guarantees of interpretability and correctness. TODO:CITE — rule-based procedure tracking or plan recognition for training (e.g., plan recognition for procedural tasks, or finite-state models of procedure execution). These approaches, however, typically require manual authoring of procedure models and do not generalise to novel learner errors or unexpected state configurations. The SimTutor system uses manually authored procedure packs (defining steps, gates, and evidence requirements) as the backbone of its state tracking, but delegates the natural-language explanation and overlay target selection to the LLM. This division of labour preserves the interpretability of the procedure model while enabling the flexibility of language-model-generated feedback.

Hybrid neuro-symbolic architectures have been proposed for task guidance, combining neural perception or generation with symbolic task models. TODO:CITE — neuro-symbolic task assistance or hybrid planning+LLM systems (e.g., SayCan-style grounding, or LLM+PDDL integration). The present work differs from these efforts in its emphasis on evidence grounding: every actionable output of the system must be traceable to a specific, auditable evidence source (a telemetry variable, a visual fact observation, a gating rule evaluation, or a retrieved document chunk). This constraint is not merely an architectural preference but a safety requirement: in the context of cockpit procedures, an incorrectly highlighted control or a misidentified procedure state could confuse or mislead the learner.

2.6 Foundation Models and VLMs for Situated Assistance

The adaptation of vision-language models (VLMs) to domain-specific structured extraction tasks is a rapidly growing area. Parameter-efficient fine-tuning methods, particularly Low-Rank Adaptation (LoRA), have made it feasible to adapt large pre-trained models to narrow domains with modest computational resources and small curated datasets. TODO:CITE — LoRA (Hu et al., 2021); TODO:CITE — QLoRA or other PEFT methods used for VLM fine-tuning; TODO:CITE — representative work on fine-tuning VLMs for domain-specific visual question answering or structured output tasks.

The VLM adaptation pipeline in this thesis (ğ**??**) follows a capture-prelabel-review-train-benchmark loop that is notable for its use of human-reviewed small data (hundreds of images, not tens of thousands) to drive meaningful improvements in structured extraction accuracy. This contrasts with large-scale VLM fine-tuning efforts that rely on web-scale data and broad instruction tuning. TODO:CITE — work on small-data domain adaptation of VLMs or LLMs, few-shot task-specific fine-tuning.

Prior work on visual understanding of graphical user interfaces and structured screen content has explored tasks such as widget detection, screen summarisation, and UI state classification. TODO:CITE — UI understanding or screen parsing with VLMs (e.g., Screen2Words, Widget Captioning, or vision-based UI automation). The cockpit display domain studied here shares structural similarities with GUI understanding — fixed regions, structured layouts, symbolic content — but differs in the criticality of the task and the need for explicit, auditable evidence rather than open-ended descriptions.

The bilingual SFT strategy used in this work (training both English and Chinese variants from the same images and labels) is related to cross-lingual transfer and instruction diversity in fine-tuning. TODO:CITE — cross-lingual instruction tuning or multilingual SFT for VLMs. However, the primary motivation here is not multilingual deployment but rather instruction diversity as a form of regularisation: by training on two languages, the model is encouraged to attend to the visual content rather than to language-specific priors, while the structured JSON output format remains language-independent.

2.7 Explainable and Grounded AI Assistance in AR/VR

The original thesis proposal envisioned an in-headset augmented reality / virtual reality tutoring interface running on the Meta Quest 3. While the current implementation

operates on a desktop DCS runtime (see ğ1.3), the long-term research agenda remains anchored in immersive procedural training.

Research on augmented reality guidance for procedural tasks has demonstrated the value of spatially registered instructions for assembly, maintenance, and repair tasks. TODO:CITE — AR-guided procedural training survey (e.g., AR for assembly, maintenance, or medical procedure guidance). These systems typically use pre-authored content (arrows, text, animations) registered to known physical objects. The SimTutor vision differs in that the instructional content is generated at runtime from the current procedure state and visual evidence, rather than retrieved from a fixed library.

Work on explainable AI in decision-support systems has emphasised the importance of traceable reasoning, particularly in safety-critical domains. TODO:CITE — explainable AI for decision support or safety-critical AI assistance (e.g., XAI in aviation, or explainable recommenders for high-stakes tasks). The evidence-grounding constraint in SimTutor aligns with this principle: each overlay target and recommended step must carry an evidence reference that links the output to its source (a telemetry variable, a gating rule, a visual fact, or a retrieved document chunk). This audit trail is designed to support both runtime debugging (why did the system highlight this control?) and post-hoc analysis of help cycles.

TODO:CITE — any work on LLM-based or AI-driven assistance in VR/AR that addresses grounding, evidence, or procedural correctness (as opposed to open-ended conversation).

2.8 Positioning of This Work

The SimTutor system occupies a position at the intersection of several research areas that are rarely combined in a single system: it applies a **telemetry-grounded** approach rather than a vision-only or text-only approach to procedural state tracking; it uses **parameter-efficient VLM adaptation** to produce structured visual facts rather than free-form descriptions; it enforces **explicit evidence grounding** on all actionable outputs; and it treats the **procedure pack** as a first-class configuration artifact that can be authored, versioned, and replayed independently of the model.

From the perspective of intelligent tutoring systems, SimTutor contributes a design in which the tutoring logic is distributed across three layers: a deterministic pack-driven procedure engine for state tracking, an LLM for grounded help generation, and a fine-tuned VLM for display-page visual fact extraction. From the perspective of VLM

adaptation research, it contributes a domain-specific benchmark and ontology evolution narrative (8-fact to 13-fact) that makes explicit the gap between early visual targets and the granularity required for reliable procedure-level reasoning. From the perspective of simulator-based training, it contributes a reusable instrumentation and replay infrastructure that can support future human-subject studies without requiring real-time experimenter oversight.

The work presented in this thesis represents a technical baseline rather than a completed evaluation. The VLM adaptation results (ğ6) provide evidence that structured visual fact extraction can be substantially improved through small-data domain fine-tuning, but they do not yet constitute evidence of learning gains, workload reduction, or transfer. The human-subject evaluation envisioned in the original thesis proposal remains a planned next phase (ğ6.2), and the current system is best understood as the runtime and instrumentation infrastructure that makes that evaluation feasible.

3 System Design

This chapter describes the design of SimTutor, a telemetry-grounded tutoring runtime for procedural training in the DCS F/A-18C cold-start task. It covers the system architecture, component design, and design rationale. Detailed methodology—including the VLM adaptation protocol, dataset curation, and benchmark design—is deferred to Chapter 4; implementation-level details of specific modules appear in Chapter 5. Section 3.1 derives the design goals from the thesis problem statement. Section 3.2 presents the overall architecture and the separation of core tutoring logic from simulator-specific adapters. Section 3.4 describes the runtime data flow through the main help cycle. Sections 3.5 and 3.6 cover the telemetry processing pipeline and the procedure and gating rule engines. Section 3.7 details the simulator adapter layer and Lua-side integration. Section 3.8 describes the event logging, replay, and scoring infrastructure. Section 3.9 presents the optional VLM adaptation component for visual state understanding. Section 3.10 concludes with a systematic comparison to the original thesis proposal, documenting how the design evolved during implementation.

3.1 Design Goals and Requirements

The thesis addresses the problem of providing grounded, context-aware procedural guidance to a learner operating a complex simulator. In the target scenario—the F/A-18C cold-start procedure in DCS World—the learner must execute a fixed sequence of 25 steps (S01–S25) spanning six phases (P1–P6), from battery power-up through engine start, avionics configuration, flight-control checks, and taxi preparation. Each step has specific preconditions that must be satisfied before execution and completion conditions that must be met before advancing. Mistakes include omissions, order errors, parameter-setting errors, and state violations—categories formalised in the error taxonomy described in Chapter 4.

From this problem, we derive the following design goals:

G1 – Simulator-based procedural tutoring. The system must observe the learner's ac-

tions through simulator state, determine which procedural step the learner is currently attempting, and provide timely, grounded guidance when the learner is stuck or requests help. The tutoring output must reference concrete cockpit elements and cite evidence from telemetry, documentation, or visual facts.

G2 – State-aware step guidance. The system must track procedure progress through a formal state machine. It must evaluate preconditions before allowing a step to become active and verify completion conditions before advancing. Guidance must be conditional on the current simulator state, not merely on step order.

G3 – Separation between tutoring logic and simulator-specific integration. The core tutoring logic—procedure tracking, gating, scoring, and help generation—must be independent of any particular simulator, model provider, or knowledge source. Simulator-specific code, model-provider code, and knowledge-retrieval code must be isolated behind stable interfaces so that the core can be tested without a running simulator and can be ported to other simulators or training domains.

G4 – Replayability and inspectability. Every interaction between the learner, the tutoring system, and the simulator must be recorded as structured, timestamped events. These event logs must support offline replay for debugging, automated scoring against a procedure pack, and post-hoc analysis of learner behaviour. They must also support replay of telemetry captures and benchmark traces for the VLM adaptation pipeline.

G5 – Optional foundation-model and VLM support. The system must be able to operate with or without a large language model. When a model is available, it may provide richer diagnoses and explanations. When unavailable or when its output fails validation, the system must degrade gracefully to deterministic, rule-based fallback behaviour. Vision-language model (VLM) support for visual fact extraction from cockpit screenshots must be an optional, independently operable component.

G6 – Extensibility to different simulators and procedures. The procedure definition, UI target mapping, telemetry mapping, error taxonomy, and knowledge source policy must be expressed as declarative configuration (YAML files) rather than hard-coded in source code. Adding a new procedure or adapting the system to a different simulator should require only new configuration artifacts, not changes to the core tutoring logic.

These goals directly inform the architectural decisions described in the following sections.

3.2 Overall Architecture

SimTutor follows a ports-and-adapters (hexagonal) architecture. The system is organised into three main layers, each with clearly defined responsibilities and dependency directions.

Core layer (`core/`). The core contains all domain logic that is independent of any particular simulator, model provider, or external service. It defines the data contracts for runtime messaging (Section 3.4), the procedure state machine and gating rule engine (Section 3.6), the scoring and interaction-metrics engines, the vision fact ontology and sticky/TTL persistence semantics, the BM25 retrieval implementation, the HelpResponse schema builder, and the security and audit infrastructure. By design, the core depends only on standard Python libraries and the abstract interfaces defined in the ports layer.

Ports layer (`ports/`). The ports layer defines stable, technology-agnostic Python protocols for the four external dependencies the core may need at runtime: model reasoning (`ModelPort`), knowledge retrieval (`KnowledgePort`), vision input (`VisionPort`), and telemetry input (`TelemetryPort`). These interfaces are the only dependency injection points between the core and the outside world.

Adapters layer (`adapters/`). The adapters layer contains all simulator-specific, provider-specific, and infrastructure-specific code. It includes DCS-BIOS telemetry receivers, the telemetry enrichment and delta-sanitisation pipeline, DCS overlay senders, OpenAI-compatible model clients (with and without multimodal support), Ollama fallback clients, a deterministic model stub for testing, a local BM25 knowledge adapter, the vision fact extraction runtime, and the response-mapping and action-execution components that bridge model output to runtime behaviour. Adapters depend on the ports they implement and on the core types they produce and consume; they never introduce reverse dependencies into the core.

The three layers are complemented by two further structural elements:

Procedure packs (`packs/`). A procedure pack is a directory of declarative YAML configuration files that fully describe one procedural task. The pack defines the step

list, preconditions and completion conditions (gating rules), UI targets and their simulator-specific element identifiers, telemetry variable mappings, error taxonomy categories and scoring weights, delta-sanitisation policy, vision fact ontology with step bindings, and vision layout description. The current repository contains one pack: `fa18c_startup`, covering the full F/A-18C cold-start procedure.

CLI and runtime entrypoints (`simtutor/`, `live_dcs.py`). The `simtutor` package provides a command-line interface for running mock scenarios, replaying and validating event logs, scoring runs, recording VLM predictions for benchmarking, and replaying DCS-BIOS raw captures. The `live_dcs.py` module orchestrates the full live runtime loop: it wires together a DCS-BIOS receiver, the telemetry enrichment pipeline, knowledge retrieval, model-based help generation, optional VLM visual fact extraction, overlay action execution, and event logging into a single long-running process.

Figure 3.1 shows the high-level component diagram.

Figure 3.1: SimTutor system architecture. The core layer contains simulator-agnostic domain logic. The ports layer defines abstract interfaces. The adapters layer provides concrete implementations for DCS, model providers, knowledge sources, and the vision pipeline. Procedure packs supply declarative configuration.

3.3 Separation of Core Tutoring Logic and Simulator Adapters

The separation between simulator-independent core logic and simulator-specific adapters serves four concrete purposes:

1. **Testability.** The core logic can be tested with mock adapters and deterministic scenarios without requiring a running DCS instance, a GPU, or network access to a model server. The repository includes a `MockEnvAdapter` that replays pre-recorded observation sequences and a `ModelStub` that returns deterministic responses, enabling fast, reproducible unit and integration tests.

2. **Maintainability.** When a simulator API changes, only the affected adapter needs to be updated; the core remains unchanged. When a model provider changes (e.g.,

from Ollama to an OpenAI-compatible vLLM server), only the model adapter configuration is affected. Three providers (`openai_compat`, `ollama`, `stub`) are supported through a single configuration interface.

3. **Transferability.** Porting the system to a new simulator or aircraft requires writing new telemetry and overlay adapters and creating new procedure packs. The procedural reasoning, gating, scoring, and help-generation infrastructure can be reused as-is. The port interfaces are intentionally minimal: a telemetry port requires `start`, `poll`, and `stop` methods; an overlay port requires only the ability to send highlight, clear, and pulse commands to named cockpit elements.

4. **Independent evolution of the VLM pipeline.** The VLM visual fact extraction component communicates with the tutoring runtime through the vision port and the vision fact observation schema, not through direct coupling. The VLM pipeline can be developed, trained, and benchmarked independently of the tutoring runtime. A runtime deployment can include the VLM component, omit it entirely, or replace it with a different perception backend without affecting the tutoring logic.

3.4 Data Flow

The runtime data flow follows a structured pipeline from simulator observation to tutoring output and logging. Figure 3.2 provides a visual summary.

Figure 3.2: Main runtime data flow through the help cycle.

3.4.1 Observation

The simulator adapter—in the current implementation, a DCS-BIOS UDP receiver—produces a stream of *observations*: versioned, timestamped data objects containing raw simulator state and, after enrichment, a set of normalised state variables.

3.4.2 Tutor Request

When the learner triggers help—via a hotkey or a UDP message—a *tutor request* is created. The request carries an intent (`help`), an optional free-text message, a reference to the current observation, and a context dictionary populated by the runtime orchestrator.

3.4.3 Context Assembly

The runtime orchestrator assembles a context for the help cycle by gathering:

- **State variables** (`vars`): normalised key–value pairs from the most recent telemetry observation.
- **Recent deltas** (`recent_deltas`): a sanitised, aggregated summary of recent state changes.
- **Candidate steps** (`candidate_steps`): eligible step identifiers from the procedure pack.
- **Deterministic step hint** (`deterministic_step_hint`): a locally computed fallback suggestion from rule-based inference, used both as prompt guidance and as fallback when model output is unavailable.
- **Overlay target allowlist** (`overlay_target_allowlist`): the set of UI elements the model is permitted to reference.
- **Knowledge snippets** (`rag_topk`, optional): top-k BM25 retrieval results, included only when a knowledge index exists.

3.4.4 Model Inference (Optional)

If a ModelPort implementation is available, the assembled context is formatted into a prompt and sent to the language model. The model returns a structured JSON object conforming to the HelpResponse schema, which is generated dynamically at runtime from the procedure pack, UI map, and taxonomy.

3.4.5 Response Validation and Mapping

The raw model output passes through a validation pipeline: (1) JSON extraction with minimal repair, (2) schema validation, (3) allowlist checks on diagnosis step IDs, next step IDs, and overlay targets, (4) evidence-grounding validation. If validation succeeds, the help object is mapped to a `TutorResponse` with overlay actions resolved through the UI map. If validation fails, or if no model is available, the system enters deterministic fallback.

3.4.6 Action Execution

The tutor response may contain overlay actions (highlight, clear, or pulse) targeting specific cockpit elements. The action executor validates each action against the UI target allowlist, enforces a limit on concurrent highlights (default: one), and dispatches the command to the simulator via UDP. Auto-click and control-injection actions are forbidden by design.

3.4.7 Event Logging

Every stage of the help cycle is recorded as a versioned, timestamped `Event` object and appended to a JSONL event log, which serves as the authoritative record for all downstream analysis, replay, and scoring.

3.5 Telemetry Processing

The telemetry processing subsystem transforms raw simulator data into the normalised state representation through four stages:

1. **Raw ingestion.** The DCS-BIOS receiver listens on a UDP socket for binary DCS-BIOS export frames, decoding each into a structured JSON object with wall-clock timestamps and monotonically increasing sequence numbers.

2. **Variable resolution.** A telemetry map specifies how raw BIOS fields are projected onto normalised state variables. The variable resolver produces a flat dictionary of key–value pairs and tracks unavailable or unknown sources in a `vars_source_missing` set, enabling downstream components to distinguish "the value is zero" from "the value is unknown."

3. **Delta sanitisation.** A configurable policy (thresholds, debounce windows, per-variable filtering rules) suppresses spurious changes caused by switch bounce, sensor drift, and UDP out-of-order delivery.

4. **Delta aggregation.** Sanitised deltas are accumulated over a configurable time window and compressed into a compact summary for the prompt context.

The telemetry pipeline runs continuously, independent of the help cycle.

3.6 Procedure Engine and Gating Rule Engine

3.6.1 Procedure Engine

The procedure engine maintains a finite-state machine over the ordered sequence of steps. Each step is in one of four states: `pending`, `active` (at most one), `done`, or `blocked`. Operations include `activate_next`, `complete_active`, `block_active`, `rewind`, and `check_timeout`. All transitions are recorded as internal events and propagated to the event log.

3.6.2 Gating Rule Engine

The gating rule engine evaluates precondition gates and completion gates using a v1 DSL with four operators: `var_gte`, `arg_in_range`, `flag_true`, and `time_since`. The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason rather than silent treatment as false. Rules short-circuit on first failure; the failure index and reason feed into the deterministic step hint.

3.6.3 Deterministic Step Inference

When no model is available or model output fails validation, the system falls back to deterministic step inference, which examines the active step, unsatisfied gating rules, and recent UI targets to produce a text hint restricted to the currently active step. By default, no overlay highlights are sent in fallback mode unless a local rule can prove a unique, valid target given the current telemetry state.

3.6.4 Procedure Pack Structure

The procedure pack is the declarative configuration driving the entire tutoring runtime. Each step is classified by primary evidence source:

- **BIOS-priority steps** (S01, S03–S07, S09–S13, S21): completion evidence primarily in telemetry variables.
- **Vision-priority steps** (S08, S15, S18): require visual confirmation through cockpit displays.

- **Manual or out-of-layout steps** (S02, S14, S16, S17, S19, S20, S22–S25): cannot be fully observed through either telemetry or the current visual layout.

The pack also defines per-step gating rules, UI targets, tutor prompts, and profile overrides (airfield vs. carrier).

3.7 Simulator Adapter Layer

3.7.1 Telemetry Input Adapter

Receives DCS-BIOS state data over UDP. The Python-side `DcsBiosRawReceiver` parses the binary protocol and emits raw frames as JSONL records, which feed the telemetry enrichment pipeline.

3.7.2 Overlay Output Adapter

Sends highlight, clear, and pulse commands to DCS via plain-text UDP messages. A Lua script (`VRHilite.lua`) inside DCS renders graphical overlays on cockpit elements. The adapter maps abstract target names to DCS-internal point codes using the procedure pack's UI map.

3.7.3 Vision Input Adapter

Triggers screenshot captures in DCS via a UDP-based capture trigger. The Lua-side `SimTutor.lua` renders the three primary cockpit displays into a single composite image. The Python-side adapter selects pre-trigger and trigger frames and passes them to the vision fact extractor.

3.7.4 Lua-Side Integration

Three categories of Lua scripts execute inside DCS:

- **State export** (`Export.lua`): configures DCS-BIOS export.
- **Overlay rendering** (`Hooks/VRHilite.lua`, `Hooks/SimTutorHighlight.lua`): receives and renders highlight commands.

- **SimTutor integration** (`SimTutor/SimTutor.lua`, `SimTutor/SimTutor Function.lua`, `SimTutor/SimTutorDcsBiosHub.lua`): handles capture and composite-panel rendering.

3.7.5 Planned and Partially Implemented Adapters

The current repository includes a `MockEnvAdapter` for deterministic testing without a running simulator. The original thesis proposal envisioned a Meta Quest 3 VR adapter. This adapter has not been implemented; the VR deployment remains planned (Section 6.2).

3.8 Event Logging and Replay

3.8.1 Event Model

All runtime occurrences are represented as `Event` objects with a fixed schema: unique identifier, wall-clock timestamp, session identifier, event kind (`observation`, `tutor_request`, `tutor_response`, `overlay_requested`, `overlay_applied`, `overlay_failed`), typed payload, schema version, and extensible metadata.

3.8.2 JSONL Event Store

Events are persisted in JSONL format via `JsonlEventStore`, supporting append-only writing and full-file loading.

3.8.3 Replay and Validation

A recorded event log can be replayed offline to verify step ordering and state machine consistency. Telemetry logs are independently validated for monotonic sequence numbers and timestamps. The replay infrastructure has been implemented at the code level and exercised with recorded telemetry traces; it has not yet been validated with end-to-end human-in-the-loop sessions.

3.8.4 Scoring and Interaction Metrics

The scoring engine counts omissions and state violations with configurable per-category weights and a critical-step multiplier. Interaction metrics characterise learner behaviour:

stall time, help requests, LLM triggers, UI click counts, and overlay request/acknowledgement counts.

3.8.5 Current Limitations

The replay infrastructure currently supports synthetic regression replay at the telemetry level, including scenarios with vision frame sidecars (`replay_eval/fa18c_startup_v04/`). End-to-end replay of full multimodal sessions with synchronised vision frames from live human runs has not yet been validated.

3.9 Optional VLM Adaptation for Visual State Understanding

Visual perception in SimTutor is an *optional, independently operable* subsystem.

3.9.1 Role of the VLM Component

The VLM component extracts structured *visual facts* from cockpit screenshots—intermediate propositions about visible display content. Visual facts are not final tutoring decisions; they provide visual evidence that the tutoring runtime combines with telemetry, procedure state, and documentation. This decomposition enables each component to be developed, tested, and evaluated independently.

3.9.2 Vision Fact Ontology

The current ontology (v2) defines 13 facts spanning three display regions:

- **Left DDI**: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_vi`
- **Right DDI**: `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_vi` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD**: `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`.

Each fact has a three-valued state (`seen`, `not_seen`, `uncertain`), a time-to-live, an optional `sticky` flag, and step bindings via `all_of` / `any_of` conditions.

3.9.3 VLM Integration in the Help Cycle

When a help request arrives and the active step is vision-priority (S08, S15, S18), a capture trigger is sent to DCS. Pre-trigger and trigger frames are rendered and sent to the VLM. The returned facts are merged into the vision fact snapshot (with sticky and TTL semantics) and included in the help-cycle context. If the vision adapter is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only.

3.9.4 VLM Adaptation Pipeline

The VLM used at runtime is fine-tuned. The adaptation pipeline—detailed in Chapter 4—comprises screenshot capture, pre-labeling by a large VLM, human review in Label Studio, bilingual SFT export, LoRA fine-tuning (Unsloth + PEFT + TRL), and base-vs-LoRA benchmarking on independent held-out sessions. This pipeline forms a self-contained contribution spanning data capture through benchmark evaluation.

3.9.5 Distinction from Rule-Based State Gating

Rule-based gating evaluates telemetry variables deterministically; it is deterministic and auditable but limited to telemetry-representable conditions. VLM-based visual fact extraction interprets display content that has no telemetry representation. The two mechanisms are complementary: the gating engine provides the primary procedure-tracking backbone; visual facts supply supplementary evidence for steps whose completion cannot be inferred from telemetry alone.

3.10 Differences from the Original Proposal

The system described in this chapter differs in several important respects from the system envisioned in the original thesis proposal. We document these differences to establish a clear boundary between the original research vision and the current implemented system.

3.10.1 From VR-First to Desktop-First Runtime

The original proposal described a Meta Quest 3 VR tutoring system with in-headset step cards. The current implementation is a desktop/DCS runtime with cockpit overlay highlighting as the primary output modality. This shift occurred because building a

telemetry-grounded runtime with delta sanitisation, deterministic fallback, and automated tests proved a substantial prerequisite for VR deployment, and the desktop configuration provides a more controlled environment for systematic benchmarking. The VR deployment remains a planned next phase (Section 6.2).

3.10.2 Added VLM Adaptation Pipeline

The original proposal envisioned a text-only RAG+LLM tutor. The VLM visual fact extraction pipeline was added after discovering that several critical procedure steps require visual confirmation not inferable from telemetry alone. The VLM pipeline evolved into an adaptation and benchmarking loop that now constitutes a substantial technical contribution.

3.10.3 Expanded Telemetry Grounding

The current implementation includes a substantially more elaborate telemetry subsystem: raw DCS-BIOS UDP ingestion, variable resolution with source-missing tracking, delta sanitisation, and delta aggregation. This expansion was driven by the practical realities of working with noisy, high-frequency simulator telemetry.

3.10.4 Added Procedural, Replay, and Benchmark Infrastructure

The original proposal did not describe a formal procedure engine, gating rule engine, error taxonomy, event logging, replay, or automated benchmarking. These were developed during implementation as essential infrastructure for systematic evaluation.

3.10.5 Postponed Human-Subject Evaluation

The original proposal planned a three-condition human-subject study that has not been conducted. The current repository contains the technical infrastructure to support such a study but no participant data. The human-subject evaluation remains the central planned component (Section 6.2).

3.10.6 Summary of Design Evolution

Table 3.1 summarises the key differences.

Table 3.1: Design evolution from proposal to implemented system.

Aspect	**Original Proposal**	**Implemented System**
Platform	Meta Quest 3 VR	Desktop / DCS
Output modality	In-headset step cards	Cockpit overlay highlights
State input	Read-only DCS state flags	Full DCS-BIOS telemetry with delta sanitisation
Visual perception	None (text-only RAG)	Optional VLM visual fact extraction (13-fact v2)
Model adaptation	Not addressed	LoRA fine-tuning pipeline (primary line: Qwen3.5-9B; supplementary: Gemma-4-31B)[1]
Procedure representation	Implicit in prompts	Explicit pack-driven configuration with gating rules
Error handling	Not specified	Structured taxonomy, deterministic fallback, and action restrictions
Evaluation infrastructure	Not specified	Event logging, replay validation, automated scoring, benchmark pipeline
Human-subject study	Planned as main evaluation	Postponed; infrastructure prepared

This evolution does not invalidate the original proposal. Rather, the implemented system provides a concrete technical foundation—with running code, automated tests, and benchmark artifacts—on which the planned VR deployment and human-subject evaluation can be built.

[1]Qwen3.5-9B (Qwen/Qwen3.5-9B-Base) fine-tuning and benchmark results are the primary verified experimental line (C-VLM-04, C-VLM-05). Gemma-4-31B (google/gemma-4-31B) results are supplementary (C-VLM-06, *likely*).

4 Methodology

4.1 Procedure Runtime and Telemetry Grounding

4.2 Knowledge Grounding and Source Policy

4.3 Visual Fact Data Pipeline

4.4 Dataset Curation

4.5 LoRA Fine-Tuning Configuration

4.6 Benchmark Protocol

5 Implementation

5.1 Technology Stack

5.2 Core Domain Logic

5.3 Telemetry and DCS Integration

5.4 Structured Help Generation

5.5 VLM Visual Fact Extraction

5.6 Replay and Logging Infrastructure

5.7 CLI and Runtime Entrypoints

6 Evaluation

This chapter presents the empirical evidence gathered during the development of the visual fact extraction subsystem and the runtime infrastructure that supports it. It is divided into two clearly delineated parts. Part A (Section 6.1) reports technical validation results that are backed by benchmark data, dataset statistics, and training artifacts currently present in the repository. Part B (Section 6.2) describes the planned human-subject evaluation that constitutes the original research vision of this thesis but has not yet been conducted; it is included here to document the evaluation design against which future work can be measured.

6.1 Current Technical Results

The technical evaluation addresses a single core question: can a LoRA-fine-tuned vision-language model (VLM) reliably extract structured visual facts from F/A-18C cockpit screenshots, and does the resulting adapter substantially outperform the untuned base model? The question is assessed through a series of benchmark comparisons on data that the model has never seen during training (holdout sets with verified zero overlap). The metrics reported below are computed by the automated benchmark harness described in Section 4.6 and are drawn directly from the benchmark artifacts stored in the repository.

6.1.1 Dataset and Training Summary

The visual fact extraction task evolved across two generations of fact ontology. The first generation (v1) defined 8 visual facts and was used to establish the initial training and evaluation pipeline. The second generation (v2) expanded the ontology to 13 visual facts, aligning it with the runtime visual contract defined in `packs/fa18c_startup/vision_facts.yaml`. The 13 facts cover panel-level page visibility (e.g., `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`), fine-grained state indicators within panels (`fcsmc_intermediate_result_visibl` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`), and small-text recognition targets (`ins_grnd_alignment_text_visible`, `ins_ok_text_visible`).

Table 6.1 summarises the six annotated datasets present in the repository, each with human-reviewed ground-truth labels.

Table 6.1: Summary of annotated visual fact datasets. Bilingual export produces separate EN and ZH samples from the same reviewed tasks. Holdout sets are English-only.

Dataset	Run	Facts	Tasks	Bilingual	Role
`vision_sft`	Run-001	8	180	yes	v1 train
`vision_sft_holdout_run002`	Run-002	8	50	no	v1 hold
`vision_sft_holdout_run002_newfacts`	Run-002	13	50	no	v2 hold
`vision_sft_run003`	Run-003	13	220	yes	v2 training
`vision_sft_run005_composition_rebalance`	Run-005	13	122	yes	v2 rebal
`vision_sft_holdout_run004_random`	Run-004	13	100	no	v2 hold

The v2 training recipe combines Run-003 (220 reviewed tasks) with Run-005 (122 reviewed tasks, collected specifically to rebalance under-represented fact states) and applies Run-005 twice (hereafter Run-005×2) as a moderate oversampling strategy. Bilingual export (EN + ZH) of Run-003 (220×2 = 440 rows) and Run-005×2 (122×4 = 488 rows) yields a combined training set of **928 rows**. The v2 holdout sets—Run-002 newfacts (50 samples) and Run-004 random (100 samples)—are English-only and have been verified to contain **zero exact overlap** with the training data.

6.1.2 Qwen3.5 8-Fact V1 Baseline

The earliest complete benchmark line uses an 8-fact LoRA adapter trained on the Run-001 dataset (180 tasks) and evaluated on the Run-002 holdout set (50 samples). This line predates the 13-fact ontology and serves primarily as a historical reference point; it is included here to establish the performance trajectory but may be more appropriately placed in the appendix.

Table 6.2 shows a substantial improvement in fact-level metrics: fact accuracy rises from 0.7600 to 0.9150, and seen F_1 nearly doubles from 0.4714 to 0.8380. However, the critical false positive count increases from 13 to 15, indicating that the 8-fact v1 adapter did not suppress hallucination on high-risk facts (facts whose false-positive assertions could mislead the tutoring system into skipping required steps). The sample exact match rate, at 0.36 for the LoRA adapter, shows that fully correct extraction of all eight facts within a single screenshot remained challenging.

Table 6.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	Base	LoRA-v1	Δ
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

6.1.3 Qwen3.5 13-Fact V2 Results

The primary technical result of this work is the performance of the Qwen3.5-9B LoRA adapter trained on the Run-003 + Run-005×2 recipe under the 13-fact v2 ontology. The adapter was trained for 4 epochs at a learning rate of 2×10^{-4} with LoRA rank $r = 16$, $\alpha = 16$, dropout = 0.0, effective batch size 4, and maximum sequence length 4096. Training converged in 6419 seconds with a final training loss of 0.2156.

Table 6.3 reports results on two held-out evaluation sets: the Run-002 newfacts holdout (50 samples) and the Run-004 random holdout (100 samples). Both holdouts are English-only and have zero exact overlap with the training data.

Table 6.3: 13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA (Run-003 + Run-005×2) on two holdout sets.

	Run-002 newfacts (50)			Run-004 random (100)		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.96	1.0	+0.04
Schema valid	1.0	1.0	—	0.96	1.0	+0.04
Fact accuracy	0.8677	**0.9908**	+0.1231	0.8038	**0.9931**	+0.1892
Macro F_1	0.4513	**0.6515**	+0.2002	0.4393	**0.6100**	+0.1707
Seen F_1	0.5022	**0.9648**	+0.4626	0.5659	**0.9159**	+0.3500
Exact match	0.10	**0.88**	+0.78	0.03	**0.92**	+0.89
Critical FP	11	**4**	−7	70	**8**	−62

Run-002 newfacts holdout. The LoRA adapter achieves a fact accuracy of 0.9908, a seen F_1 of 0.9648, and a sample exact match rate of 0.88. This means that in 44 out of 50 samples, all 13 facts are correctly classified. The base model, in contrast, achieves a fact accuracy of 0.8677 and an exact match rate of only 0.10. The critical false positive count

drops from 11 (base) to 4 (LoRA), a reduction of 7. The three remaining error sources in the LoRA adapter are: two false negatives on `tac_page_visible` (the adapter misses the TAC page when it is present), one false positive on `fcsmc_final_go_result_visible`, one false positive on `ins_grnd_alignment_text_visible`, and two false positives on `ins_ok_text_visible`.

Run-004 random holdout. The pattern is consistent but the base model degrades more severely on this larger and more diverse holdout set. The base model produces 4 invalid JSON/schema responses (yielding a validity rate of 0.96), 70 critical false positives (nearly all concentrated on `fcsmc_intermediate_result_visible` with 60), and a sample exact match rate of only 0.03. The LoRA adapter restores perfect JSON and schema validity (1.0), raises fact accuracy to 0.9931, and achieves a sample exact match rate of 0.92 (92 out of 100 samples fully correct). The critical false positive count drops from 70 to 8, all of which are on `fcsmc_final_go_result_visible`.

Across both holdouts, the LoRA adapter demonstrates three key properties. First, it restores output format reliability: the base model occasionally produces malformed JSON or schema-invalid responses on the Run-004 holdout, while the LoRA adapter does not. Second, it dramatically improves per-fact recall, as captured by seen F_1 rising from the range 0.50–0.57 (base) to 0.92–0.96 (LoRA). Third, it reduces critical false positives by 64% (Run-002) and 89% (Run-004), though a residual set of false positives on completion-type facts (`fcsmc_final_go_result_visible`, `ins_ok_text_visible`) persists.

Comparison to 8-fact v1. Moving from the 8-fact v1 to the 13-fact v2 ontology, the LoRA adapter's fact accuracy rises from 0.9150 to 0.9908 on Run-002-family holdouts, and the sample exact match rate rises from 0.36 to 0.88. Critically, the v2 adapter reduces critical false positives (from 15 to 4 on Run-002-family) rather than increasing them, as the v1 adapter did relative to its base model. This indicates that the combination of the expanded fact ontology, the Run-003 + Run-005×2 data recipe, and the composition-rebalance strategy addressed the false-positive vulnerability that was present in the v1 line.

6.1.4 Gemma-4 13-Fact V2 Comparison

To assess whether the training recipe generalises beyond a single backbone architecture, the identical data recipe (Run-003 + Run-005×2, 928 rows, bilingual, 13-fact ontology) was applied to a second VLM backbone: `google/gemma-4-31B`. Training hyperparameters were matched wherever possible: 4 epochs, learning rate 2×10^{-4}, LoRA $r = 16$,

$\alpha = 16$, dropout = 0.0, effective batch size 4, maximum sequence length 4096. Differences include the use of `all-linear` LoRA target modules (vs. Qwen's vision+language attachment), 4-bit quantization (`load_in_4bit=true`), GPU memory utilisation of 0.95, and the `gemma-4` chat template. Training converged in 8050 seconds with a final training loss of 0.0474. Note that training loss values are not directly comparable across backbones due to differences in tokenizer behaviour, template structure, and parameterisation.

Table 6.4: 13-fact v2 benchmark: Gemma-4-31B Base vs. LoRA (Run-003 + Run-005×2) on two holdout sets.

	Run-002 newfacts (50)			**Run-004 random (100)**		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.99	1.0	+0.01
Schema valid	1.0	1.0	—	0.99	1.0	+0.01
Fact accuracy	0.8169	0.9492	+0.1323	0.8146	0.9715	+0.1569
Macro F_1	0.4432	0.5857	+0.1425	0.4400	0.5630	+0.1229
Seen F_1	0.5128	0.8123	+0.2995	0.6332	0.7959	+0.1627
Exact match	0.04	0.44	+0.40	0.12	0.74	+0.62
Critical FP	16	**0**	−16	47	13	−34

Table 6.4 confirms that the Run-003 + Run-005×2 data recipe produces meaningful improvements for the Gemma backbone as well. On both holdouts, fact accuracy rises from the base model's range of 0.81–0.82 to 0.95–0.97 for the LoRA adapter. The most distinctive result is on the Run-002 newfacts holdout, where the Gemma LoRA adapter achieves **zero critical false positives** (down from 16 for the Gemma base model). This contrasts with the Qwen LoRA adapter, which retains 4 critical false positives on the same holdout.

However, Gemma's conservative behaviour comes at a cost in recall and exact-match performance. On Run-002, the Gemma LoRA adapter's sample exact match rate is 0.44, compared to 0.88 for Qwen LoRA. On Run-004, the gap is 0.74 (Gemma) vs. 0.92 (Qwen). Seen F_1 follows a similar pattern: 0.81 (Gemma) vs. 0.96 (Qwen) on Run-002, and 0.80 (Gemma) vs. 0.92 (Qwen) on Run-004. The Gemma base model is a stronger starting point on Run-004 (seen F_1 of 0.6332 vs. Qwen base's 0.5659), but the Qwen LoRA adapter's improvement is larger, yielding a higher absolute performance ceiling.

The practical interpretation, consistent with the backbone comparison report, is that the Qwen3.5-9B adapter remains the better choice for downstream system integration, owing to its higher overall accuracy, stronger seen F_1, and higher sample exact match

rate. Gemma's zero-critical-FP result on Run-002 is noteworthy and suggests that backbone-level differences in error style (conservative vs. assertive) are observable even under matched training data.

6.1.5 Error Analysis

To understand where the remaining errors occur and what patterns differentiate the base model from the LoRA adapter, this section provides a per-fact breakdown of the strongest configuration (Qwen LoRA on the 13-fact v2 ontology).

Fact Categories by Difficulty

The 13 visual facts can be grouped into three categories that reflect distinct levels of extraction difficulty.

Group 1: Coarse panel presence (6 facts). Facts in this group indicate whether a particular cockpit panel or display page is currently visible: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `bit_root_page_visible`, `fcsmc_page_visible`, and `hsi_page_visible`. These are structural recognition tasks that rely on identifying large, visually distinctive panel layouts.

The Qwen LoRA adapter achieves perfect classification on all six facts on both the Run-002 newfacts and Run-004 random holdouts, with the single exception of `tac_page_visible` on Run-002 (two false negatives out of 50 samples; the TAC page is seen in 23 of 50 samples, so these represent missed detections rather than hallucinations). The base model, by contrast, fails on several of these facts: it completely misses all five occurrences of `supt_page_visible` on Run-002 (seen $F_1 = 0.0$), hallucinates five false positives on `bit_root_page_visible` on Run-002, and misses 18 of 94 occurrences of `fcs_page_visible` on Run-004 (seen recall = 0.81).

Group 2: Fine-grained state indicators (5 facts). Facts in this group capture subtle within-panel states: `fcs_page_x_marks_visible` (X-marks on the FCS page), `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible` (three progressive states of the FCS-MC test sequence), and `hsi_map_layer_visible` (whether the map overlay is active on the HSI).

The Qwen LoRA adapter achieves perfect classification on `fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, and `hsi_map_layer_visible` on both holdouts. One false negative occurs on `fcsmc_in_test_visible` on Run-004 (17 seen samples, 16 correctly identified). The most persistent error source is `fcsmc_final_go_result_visible`

where the adapter produces 1 false positive on Run-002 and 8 false positives on Run-004 (against 60 seen occurrences). This fact requires distinguishing a specific numeric readout within the FCS-MC panel, and the consistent false-positive pattern suggests that visually similar intermediate states are occasionally misclassified as the final GO result.

The base model's behaviour on this group is qualitatively different. On Run-004, the base model generates 60 critical false positives on `fcsmc_intermediate_result_visible` alone (against only 7 true seen occurrences), indicating a near-complete failure to distinguish this state. The base model also shows poor recall on `fcs_page_x_marks_visible` (seen $F_1 = 0.0$ on both holdouts) and `fcsmc_in_test_visible` (seen recall ≈ 0.33–0.53).

Group 3: Small-text recognition (2 facts). Facts `ins_grnd_alignment_text_visible` and `ins_ok_text_visible` require the model to recognise small textual indicators on the INS panel. These are the most fine-grained facts in the ontology and represent a near-OCR-level challenge for the 9B-parameter model.

The Qwen LoRA adapter performs strongly on `ins_grnd_alignment_text_visible`: perfect classification on Run-004 (69 seen, 31 not seen) and one false positive on Run-002 (42 seen, 7 not seen). On `ins_ok_text_visible`, the adapter produces 2 false positives on Run-002 (against 4 seen occurrences; these are samples where the OK text is not yet visible but the adapter asserts it is). On Run-004, where `ins_ok_text_visible` is never seen (0 of 100), the adapter correctly produces no false positives, achieving perfect accuracy.

The base model is effectively blind to both of these facts: seen $F_1 = 0.0$ on both holdouts for `ins_grnd_alignment_text_visible`, and seen $F_1 = 0.0$ for `ins_ok_text_visible` (though the base model also never sees it on Run-004, so the F_1 metric reports 0.0 by construction).

Error Taxonomy

Aggregating across both holdouts for the Qwen LoRA adapter, the observed errors can be classified into three patterns:

1. **Completion-fact false positives (majority).** The largest share of residual errors comes from `fcsmc_final_go_result_visible` and `ins_ok_text_visible`, where the adapter occasionally asserts a completion/finalisation state that is not yet present. These are classified as *critical* false positives in the benchmark because

they carry direct downstream risk: if the tutoring system believes a step is complete when it is not, it may prematurely advance the procedure.

2. **Isolated false negatives (minor).** Two false negatives on `tac_page_visible` (Run-002) and one on `fcsmc_in_test_visible` (Run-004) represent missed detections of states that are actually present. These are less severe from a tutoring-safety perspective (a missed detection triggers a redundant prompt rather than skipping a step) but indicate residual recall gaps.

3. **Small-text hallucination (minor).** One false positive on `ins_grnd_alignment_text_visible` (Run-002) represents the adapter falsely detecting the alignment status text. Given the small visual footprint of this indicator, this error is consistent with the expected difficulty profile.

Confusion Pattern Summary

Per-fact confusion matrices (available as charts in each benchmark directory) reveal a systematic pattern in the base model's errors: the base model is prone to *unidirectional hallucination* on facts that appear rarely in the training distribution. For instance, `fcsmc_intermediate_result_visible` appears in only 7 of 100 samples in the Run-004 holdout, yet the base model asserts it as `seen` in 60 additional samples where it is not present. The LoRA adapter eliminates this class of error almost entirely, suggesting that the composition-rebalance and oversampling strategy effectively counteracted the base model's tendency to over-predict frequently co-occurring fact states.

6.1.6 Runtime Infrastructure Readiness

Beyond the VLM benchmark, the repository contains evidence of a functional replay-based evaluation infrastructure. The replay evaluation suite at `replay_eval/fa18c_startup_v04/` defines five test cases (`batteryon_2min`, `Batteryon_enginGenoff_2min`, `fcs_reset_fcs_bit_2min`, `ins_2min`, `noop_2min`) with DCS-BIOS raw captures (`dcs_bios_raw.jsonl`) and per-frame sidecar files (`frames.jsonl`). This infrastructure enables replay of recorded DCS sessions for offline testing of the telemetry pipeline, knowledge retrieval, and procedure execution logic without requiring a live DCS instance.

The runtime data collection pipeline records structured events through the `core/event_store.py` module and indexes them under `logs/test_index/index.json`. At present, this pipeline captures telemetry snapshots, procedure step transitions, and VLM inference requests.

It does not yet implement the participant/session schema, quiz linkage, or NASA-TLX integration that would be required for a human-subject study; these are discussed as planned upgrades in Section 6.2.3.

6.2 Planned Runtime Data Collection and Human-Subject Evaluation

The original research vision of this thesis, as articulated in the research proposal, includes a controlled human-subject evaluation of the grounded tutoring system in a DCS procedural learning context. This evaluation has not yet been conducted. The following subsections describe the planned evaluation design as it was formulated in the proposal, together with an assessment of the upgrades required to transition the current runtime infrastructure into a data-collection-ready state. All statements in this section represent future work and are included for transparency about the intended trajectory of the research.

6.2.1 Original Evaluation Vision

The proposal envisions a between-subjects comparison of three conditions:

1. **Video + notes baseline.** Learners study a pre-recorded video walkthrough and written checklist before and during the cold-start procedure, without any integrated tutoring system.

2. **Ungrounded text-only LLM.** A language model provides step-by-step guidance via text, but without access to real-time simulator state, retrieved manual excerpts, or visual fact extraction.

3. **Grounded RAG + LLM tutor.** The full system described in Chapters 3, 4, and 5: telemetry-grounded, retrieval-augmented, with structured visual fact extraction and overlay execution.

The original target platform is the Meta Quest 3 with DCS-VR, providing in-headset guidance through step cards or overlay annotations. The current system operates on the desktop/DCS (non-VR) configuration, so a VR deployment would need to be completed before this evaluation can proceed. Target sample size is approximately $n = 5$–10 participants per condition, drawn from a beginner-to-intermediate population with limited prior F/A-18C cold-start experience.

6.2.2 Planned Evaluation Metrics

The proposal specifies a multi-dimensional evaluation framework with the following planned metrics:

- **Immediate performance:** Procedure completion rate, procedural error rate, total task time, and dead-end count (number of times the participant becomes stuck and requires external intervention).
- **Workload:** NASA Task Load Index (NASA-TLX) administered immediately after the training session.
- **Learning:** Pre-test and post-test quiz scores measuring declarative knowledge of the cold-start procedure.
- **Retention:** A delayed post-test administered 2–4 weeks after the training session to assess knowledge retention.
- **Transfer:** Performance on a variant procedure (e.g., a modified cold-start sequence) to assess whether skills generalise beyond the trained scenario.

All of these metrics remain in the planning stage; no participant-level data has been collected.

6.2.3 Required Runtime Data Collection Upgrades

To support the planned human-subject evaluation, the current runtime data collection infrastructure requires several specific upgrades:

1. **Participant and session schema.** The event store must be extended with a structured schema for participant demographics (anonymised ID, prior experience level, condition assignment), session metadata (date, duration, DCS version), and trial-level annotations.
2. **NASA-TLX integration.** A post-session instrument for administering and recording NASA-TLX responses must be integrated into the runtime, either as an in-application form or as a linked external survey with session-level traceability.
3. **Quiz linkage.** Pre-test and post-test quiz responses must be linked to participant and session identifiers to enable within-subject learning gain analysis.

4. **Anonymisation pipeline.** All participant-identifiable data must pass through an anonymisation step before storage, in compliance with standard research ethics requirements for human-subject data.

5. **VR deployment.** The runtime must be ported from the current desktop/DCS configuration to the Quest 3 / DCS-VR platform, including the in-headset overlay rendering and interaction handling described in the proposal.

None of these upgrades have been implemented at the time of writing.

6.2.4 Planned Experiment Design

The planned experiment follows a standard pre-test / training / post-test / retention design:

1. **Recruitment and screening.** Participants are recruited from a population with limited F/A-18C experience. A screening questionnaire establishes baseline DCS familiarity and assigns participants to conditions using stratified randomisation.

2. **Pre-test.** A written quiz assesses declarative knowledge of the cold-start procedure before any training exposure.

3. **Familiarisation.** A brief DCS cockpit familiarisation phase ensures all participants can locate basic controls and panels, reducing variance from platform unfamiliarity.

4. **Training trial.** Participants execute the F/A-18C cold-start procedure under their assigned condition. The system records all telemetry, procedure state transitions, help requests, VLM inference calls, and (for the grounded condition) overlay execution events.

5. **Post-test and NASA-TLX.** Immediately following the training trial, participants complete the NASA-TLX instrument and a post-test quiz identical in structure to the pre-test.

6. **Retention test.** After a 2–4 week interval, participants return for a delayed post-test and, where feasible, a second cold-start trial to assess procedural retention.

7. **Transfer task (optional).** A variant procedure is administered to assess the generality of learned skills.

This design has been specified at the proposal level but has not been piloted or executed.

6.2.5 Known Threats to Validity

Several threats to validity can be identified at the design stage, prior to any data collection:

1. **Desktop vs. VR platform gap.** The current system operates on desktop DCS, not DCS-VR on Quest 3 as envisioned in the proposal. Results obtained in either platform may not generalise to the other without additional validation.

2. **Benchmark performance does not imply learning outcomes.** The VLM benchmark results reported in Section 6.1 demonstrate that the LoRA adapter extracts visual facts accurately. However, accurate visual fact extraction is a necessary but not sufficient condition for improving human learning outcomes. A human-subject study is required to establish whether the fact extraction performance translates into measurable procedural learning gains.

3. **Small sample size.** The planned sample size of $n = 5$–10 per condition is small by the standards of controlled human-subject research and limits statistical power for detecting between-condition differences, particularly for noisy metrics such as completion time and error rate.

4. **Beginner population generalisability.** The planned participant pool consists of individuals with limited DCS experience. Results may not generalise to experienced pilots or to other simulation platforms.

5. **Single procedure.** The evaluation is designed around a single procedure (F/A-18C cold start). Whether the tutoring approach transfers to other procedural tasks within DCS or to other domains remains an open question.

6. **Experimenter effects.** In a small-n, single-site study, experimenter behaviour during familiarisation and intervention can introduce systematic bias that is difficult to separate from condition effects.

These threats do not invalidate the planned evaluation design, but they do bound the scope of conclusions that can be drawn from its eventual results. Addressing them

through careful protocol design, pre-registration of analysis plans, and explicit acknowledgement of limitations will be essential when the study proceeds.

In summary, this chapter has presented (a) technical validation results confirming that the LoRA-fine-tuned VLM adapter substantially and consistently outperforms the base model across two independent holdout sets under a 13-fact visual ontology, with the Qwen3.5-9B backbone achieving the strongest overall results; (b) evidence that the training recipe generalises to a second backbone architecture (Gemma-4-31B), though with a different precision–recall trade-off; (c) a per-fact error analysis identifying completion-fact false positives as the primary residual failure mode; and (d) a structured description of the planned human-subject evaluation, including its design, required infrastructure upgrades, and known threats to validity, all of which remain future work.DSMLparameter>

7 Discussion

7.1 Why the Project Shifted from VR-First to Telemetry/VLM-Heavy

7.2 What Current Benchmarks Do and Do Not Prove

7.3 Ontology Evolution: 8-Fact to 13-Fact

7.4 Why Human-Subject Evidence Remains Essential

7.5 Risks in Interpreting Benchmark Success as Learning Success

7.6 Limitations

8 Conclusion

8.1 Summary of Completed Work

8.2 Future Work

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

9 Appendix